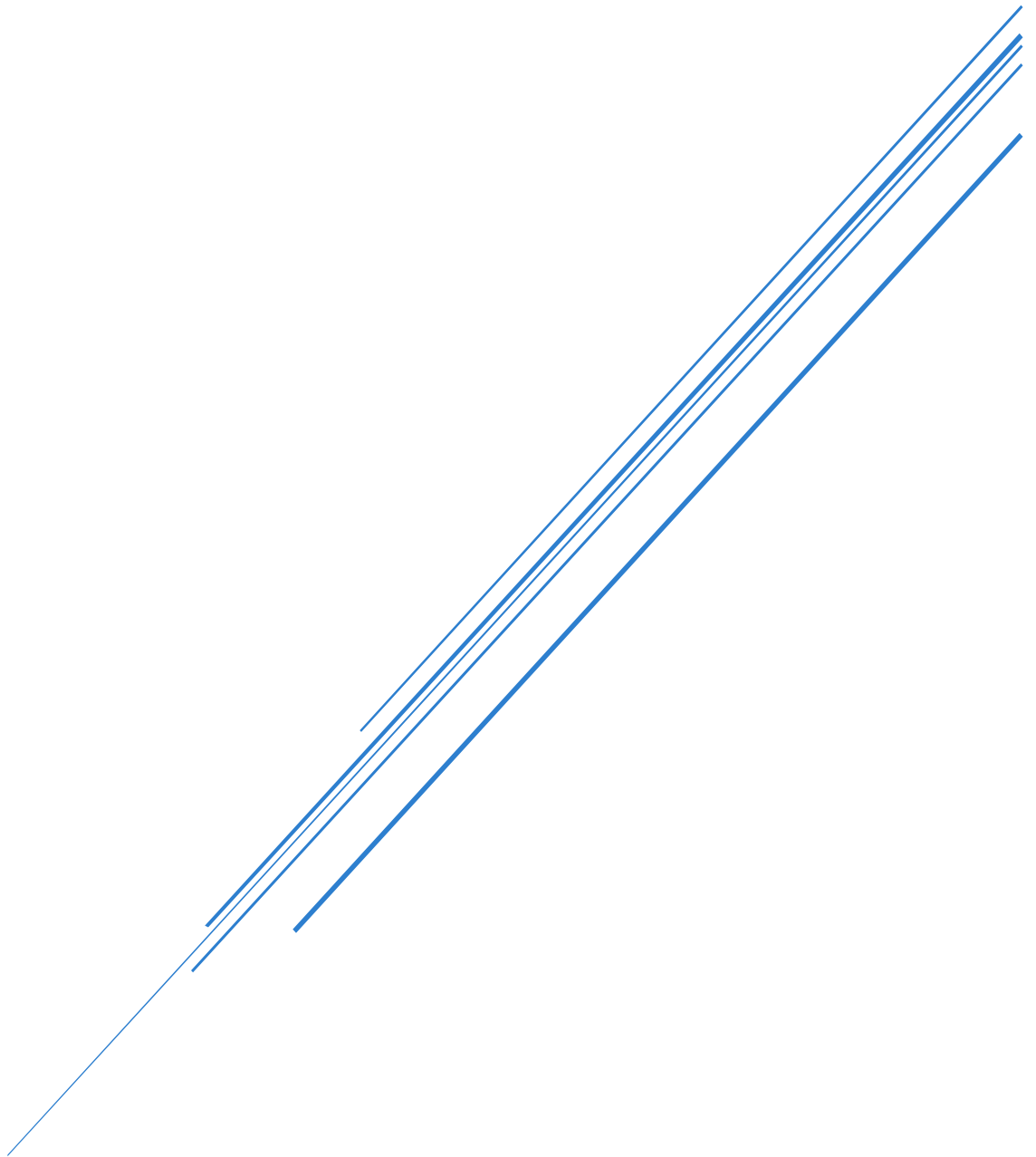


LEAGUE OF LEGENDS MACHINE LEARNING CASE STUDY

Antonio Paul Mammone – A1798933



University of Adelaide

1. Introduction

1.1 Background and Context

League of Legends (LoL) is an online team-based strategy game where two teams of five face off and the aim is to get player kills, secure objectives and take over the opponent's base to destroy their "Nexus". Within these games many metrics can determine the outcome, so it can be hard to determine the most important aspects of victory. As Esports evolves and becomes more popular the inherent use of Machine Learning can be applied to assist quantifying the importance of each variable, within this case study the aim is to find, for an individual player what some of, if not the most important aspect of their gameplay is to improve.

Within the community and this case study many terms such as CS (creep score) or KDA (Kills/Deaths/Assists) is mentioned, as prior reading to anyone unfamiliar with the game I recommend the [League of Legends Wiki](#) to get started.

1.2 Research Objective

The primary objective of this study is to develop and evaluate a binary classification system capable of predicting individual player match outcomes (Win or Loss) based on post-game statistics. Specifically, this analysis addresses three key research questions:

- **Predictive Accuracy:** Can supervised learning models achieve high accuracy (greater than 85%) in predicting match outcomes using standard in-game metrics?
- **Feature Importance:** Which gameplay factors, specifically combat mechanics versus objective control, most strongly correlate with victory?
- **Model Comparison:** How do linear classification methods compare against non-linear tree-based methods, including advanced boosting algorithms?

1.3 Dataset Overview

The analysis utilises the "[League of Legends Matches – 150k In-Game Stats](#)" dataset from Kaggle, Developed by Nathan Smallcalder. The dataset comprises match data resulting in 150,505 individual player records. Collected during Patch 25.19, the data captures a specific snapshot of the game's meta-strategy.

It is important to note that this analysis focuses on single-player performance data rather than aggregate team data. As a result, the dataset contains independent rows for individual players in any given match, rather than grouped sets of 10 players. This explains why the count of players in each lane (e.g., Top, Mid, Jungle) may vary slightly, as we are predicting the win/loss outcome for a specific player based on their personal stats, not the total match state.

The dataset is characterised by a balanced target variable, with win and loss records distributed almost evenly at 63,940 wins and 66,155 losses. This natural balance eliminates the need for synthetic resampling techniques. However, as seen in Figure 1, the rank distribution is heavily skewed towards higher competitive tiers, with Master tier players representing the largest single group. This suggests the dataset reflects skilled play patterns rather than the average user experience.

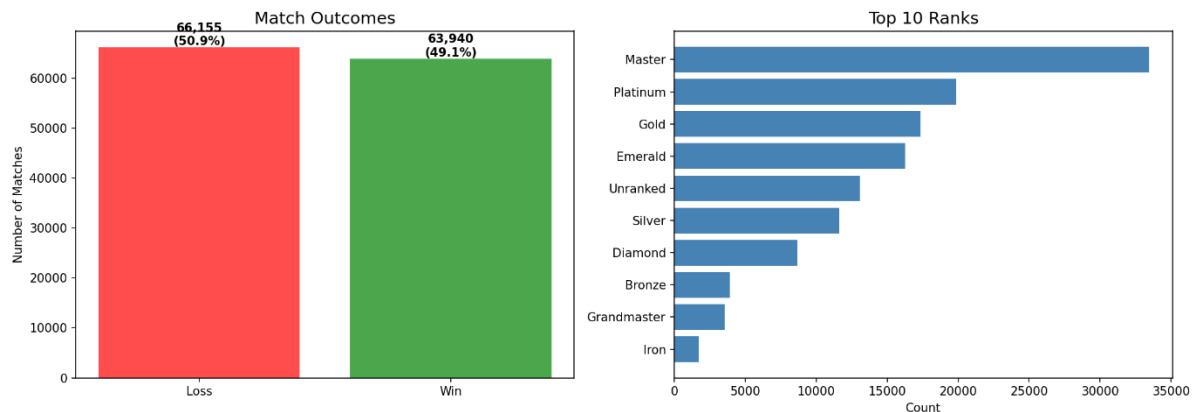


Figure 1: Distribution of Match Outcomes (Left) and Player Ranks (Right). Note the skew towards high-level Master tier play.

1.4 Methodological Approach

This study adopts the **Programming approach** to compare four classification algorithms: Logistic Regression, Decision Trees, Random Forests, and Gradient Boosting. The methodology follows a standard data science pipeline:

- **Data Integration:** Merging relational tables to consolidate player and team statistics.
- **Feature Engineering:** Creating normalised metrics to account for game duration variance.
- **Modelling:** Training classifiers using a stratified train-test split.
- **Evaluation:** Assessing performance via accuracy, F1-score, and ROC-AUC metrics.

Algorithm Selection Note: While the course curriculum focuses on standard Tree methods, **Gradient Boosting** was explicitly selected as a "Challenger" model for this study. In professional MLOps workflows, a "champion/challenger" framework is often used to test if a new, more complex model (the challenger) can statistically outperform the currently deployed baseline (the champion) before replacement (DataRobot 2023). In this study, the Random Forest serves as the baseline "champion" learned in class, while Gradient Boosting is introduced as the "challenger." While Random Forest builds trees independently (Bagging), Gradient Boosting builds them sequentially, where each

new tree attempts to correct the errors of the previous one (Scikit-learn Developers 2025; IBM 2025). This inclusion allows the study to rigorously determine if the increased computational cost of boosting yields a significant performance gain over standard course methods.

2. Methodology

2.1 Data Source and Preparation

The raw data consisted of seven relational CSV tables, necessitating a systematic merging process. The 'MatchStatsTbl' (individual performance) was joined with 'MatchTbl' (game metadata), 'RankTbl' (competitive tiers), and 'TeamMatchTbl' (team objectives) using primary and foreign keys. This integration produced a unified dataset of 150,505 observations.

Data cleaning involved handling missing values and outliers. Rows without valid lane assignments (indicating a match did not start) were removed. Initially, the missing values in optional item slots (items 4-6) caused the models to fail. I considered dropping these rows but realised that would discard nearly 15% of valid match data. Instead, I engineered a zero-imputation strategy, treating empty slots as 'No Item Purchased,' which preserved the dataset integrity.

Exploratory data analysis confirmed the dataset was balanced but highlighted significant variance in game duration, necessitating feature normalisation.

2.2 Feature Engineering

To ensure fair comparisons across matches of varying lengths, raw accumulation statistics were transformed into rates and efficiency ratios. The code in the annex details the following transformations:

- **Per-Minute Rates:** Statistics such as Gold, Creep Score (CS), and Damage Dealt were divided by game duration in minutes to create standardised performance metrics (e.g., Gold_per_min, CS_per_min).
- **Combat Ratios:** The standard KDA ratio was calculated as $(\text{Kills} + \text{Assists}) / \max(\text{Deaths}, 1)$ to quantify combat effectiveness while handling zero-death edge cases.
- **Objective Differentials:** Team-level performance was captured by calculating the difference between allied and enemy objective captures (e.g., Dragon_Differential, Tower_Differential).
- **Economic Efficiency:** Dmg_per_Gold was calculated to assess how effectively players converted economic resources into combat output.

Categorical variables such as competitive rank and lane position were label-encoded to ensure compatibility with the chosen scikit-learn algorithms.

2.3 Model Selection and Training

Four supervised learning models were selected to balance interpretability with predictive power:

- **Logistic Regression:** Selected as a linear baseline to establish minimum performance expectations.
- **Decision Tree:** Chosen for its ability to map non-linear relationships and feature interactions.
- **Random Forest:** An ensemble bagging method used to reduce the variance and overfitting risks associated with single decision trees.
- **Gradient Boosting:** As noted in section 1.4, this sequential ensemble method was employed to maximise predictive performance. By iteratively boosting weak learners, this model is often capable of capturing subtle signal patterns that Random Forests might miss.

The dataset was partitioned into a training set (80%) and a hold-out test set (20%) using stratified sampling to maintain the class balance. Feature scaling (StandardScaler) was applied specifically for the Logistic Regression model, while tree-based models were trained on unscaled data as they are scale-invariant. Hyperparameters were set to moderate values (e.g., max_depth=15, n_estimators=100) to balance model complexity with training time.

2.4 Validation Strategy

Model performance was evaluated using a comprehensive suite of metrics: Accuracy, Precision, Recall, F1-Score, and Area Under the Receiver Operating Characteristic Curve (ROC-AUC).

The ROC Curve was prioritised as a primary indicator alongside accuracy. As explored in the course case studies (e.g., Week 4 PCA Regression), the ROC curve plots the True Positive Rate against the False Positive Rate at various threshold settings. This is crucial because standard Accuracy assumes a fixed decision threshold (usually 0.5). By using ROC-AUC, this study evaluates the model's ability to rank winning probabilities correctly, regardless of where the classification threshold is set. This provides a more robust measure of separability between the 'Win' and 'Loss' classes than accuracy alone (Google 2025a).

To ensure the reliability of these results and mitigate the risk of overfitting, 5-fold stratified cross-validation was performed on the training data. This process provided

confidence intervals for the accuracy estimates, allowing for a statistical comparison of model stability.

2.5 Ethical and Bias Considerations

While the dataset excludes Personally Identifiable Information (PII), several sources of bias were identified and addressed. Firstly, sampling bias exists due to the over-representation of high-rank players (Master tier) compared to the general population. To mitigate this, rank-stratified accuracy analysis was conducted to check if the model performed poorly on lower-ranked players (Google 2025b).

Secondly, temporal bias is inherent as the data is from a single game patch (25.19). Game balance changes frequently, meaning these models would require retraining for future patches. Finally, outcome bias is present in features like Kill_Differential, which are highly correlated with winning because they are consequences of winning, not just predictors. These limitations are acknowledged when interpreting the results for practical coaching applications.

3. Results

3.1 Descriptive Analysis

Initial data exploration confirmed the dataset's suitability for binary classification. The target variable was nearly perfectly balanced (51% Loss vs 49% Win). Feature distribution analysis highlighted clear distinctions between winning and losing performances. As illustrated in **Figure 2**, winning players consistently exhibited higher Gold_per_min and KDA ratios. Interestingly, Kill Participation showed a counter-intuitive negative correlation in the summary chart. A deeper look suggests that in losing games (stomps), a single player might participate in the only two kills their team manages, resulting in 100% participation, whereas winning teams spread kills more evenly.

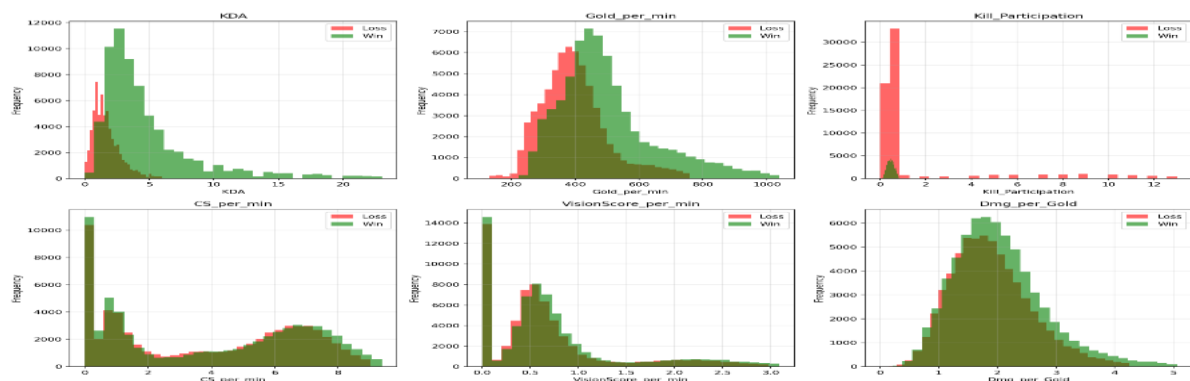


Figure 2: Feature distributions separated by Match Outcome. Winning players (Green) show consistently higher gold income and KDA ratios.

The correlation matrix in **Figure 3** further supports this, showing strong positive correlations between victory and DmgDealt_per_min, while confirming the negative relationship with Kill_Participation discussed above.

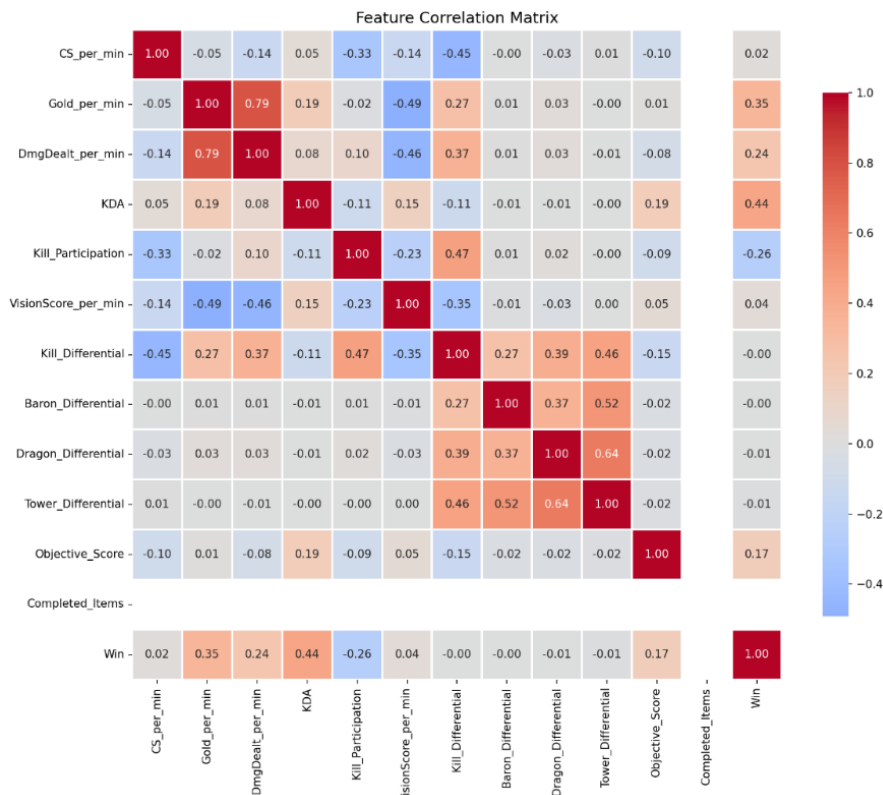


Figure 3: Feature Correlation Matrix. Note the negative correlation of Kill Participation in winning games.

3.2 Model Performance Evaluation

All four supervised learning models achieved high predictive accuracy, significantly outperforming the random baseline of 50%. The performance comparison is summarised in Figure 4.

- **Logistic Regression:** Achieved an accuracy of **84.7%** and an ROC-AUC of **0.926**. While robust, it was the weakest performer, confirming that game outcomes rely on non-linear complexities.
- **Decision Tree:** Improved accuracy to **86.4%** (ROC-AUC 0.950), demonstrating the value of non-linear splits.
- **Random Forest:** Further improved accuracy to **87.2%** (ROC-AUC 0.957).
- **Gradient Boosting:** Justified its inclusion by emerging as the superior model, achieving the highest Accuracy of **88.6%** and ROC-AUC of **0.964**.

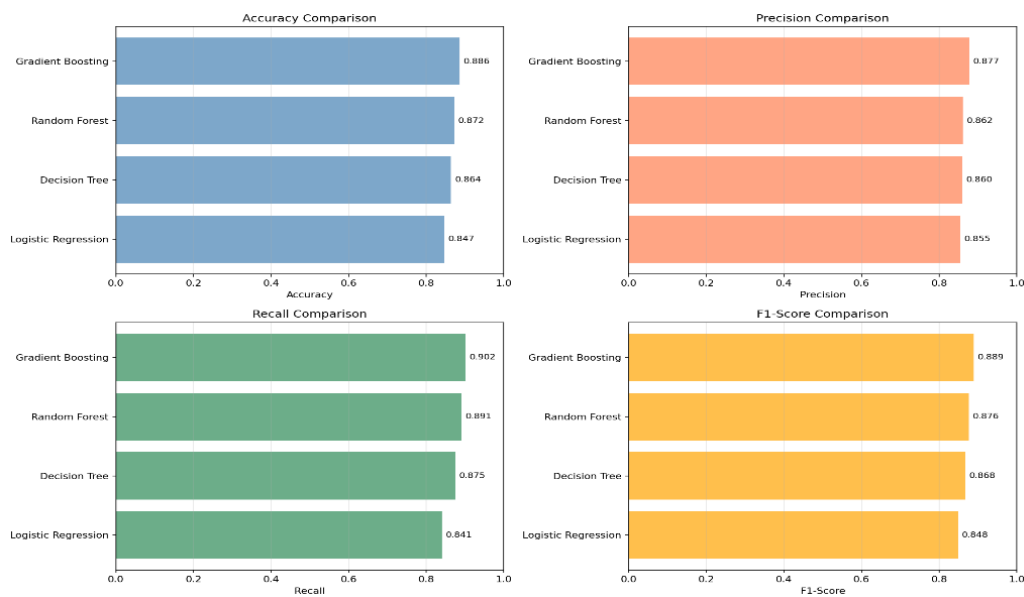


Figure 4: Model Performance Metrics Comparison. Gradient Boosting achieves the highest accuracy across all metrics.

The model evaluation is further detailed in Figure 5, which presents the ROC Curves (Left) and Confusion Matrices (Right) side-by-side to contrast overall discriminative ability with specific error types.

The ROC Curves in Figure 5a confirm Gradient Boosting's superiority, with its curve (Red) hugging the top-left corner most tightly. To further analyse prediction errors, confusion matrices were generated for all models (see Figure 5b). The Gradient Boosting model (bottom right) demonstrated the most balanced performance, correctly identifying 11,933 wins while minimizing false negatives, which is crucial for reliably predicting victory conditions.

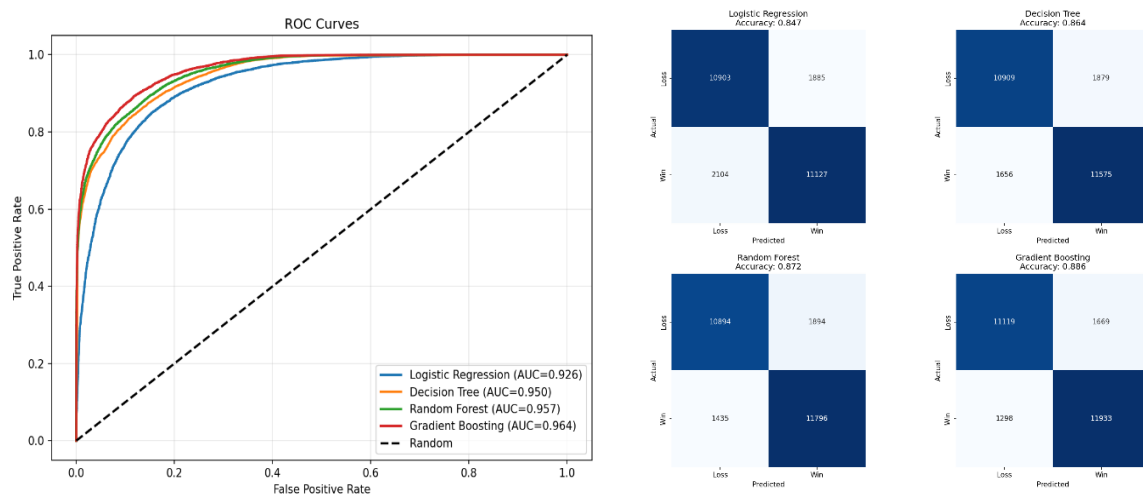


Figure 5: (a) ROC Curves showing Gradient Boosting's superior AUC of 0.964. (b) Confusion Matrices highlighting the balanced prediction accuracy of the Gradient Boosting model.

3.3 Feature Importance

The feature importance analysis provided a critical insight that contradicts the initial hypothesis that objective damage would be the primary driver. Across all tree-based models (Decision Tree, Random Forest, and Gradient Boosting), the top predictors were consistent (see Figure 6):

1. **KDA (Kill/Death/Assist Ratio):** This was overwhelmingly the highest-ranked feature. This suggests that simply *staying alive* while contributing to fights is the single best predictor of a win.
2. **Kill Participation:** Consistently ranked second.
3. **Gold Per Minute:** Ranked third.

Surprisingly, objective metrics like Tower_Differential ranked significantly lower in the importance charts. This implies a causal chain; high KDA and economic dominance (Gold) are the *causes* that allow a team to take objectives. The model identifies that if a player dominates combat, the objectives (and the win) naturally follow.

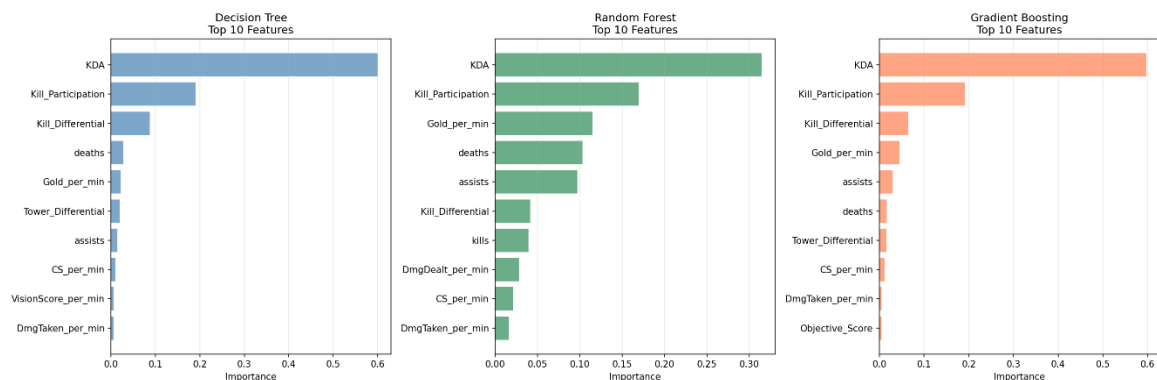


Figure 6: Feature Importance across tree-based models. KDA is consistently the dominant predictor of match outcome.

3.4 Rank-Stratified Analysis

To address sampling bias, model accuracy was evaluated across different competitive ranks. As shown in **Figure 7**, the Random Forest model showed remarkable stability, with accuracy ranging from 86% to 89% across tiers. Notably, accuracy was highest in **Emerald (89%)** and **Master (88%)** tiers. This confirms the hypothesis that high-ranked players behave more predictably according to meta-strategies, whereas lower-ranked games (Iron/Bronze) contain more erratic behaviour that is slightly harder to model.

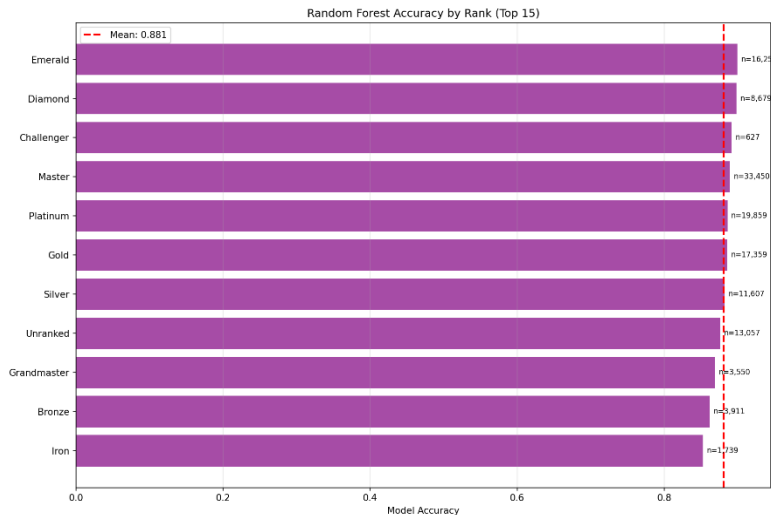


Figure 7: Random Forest Accuracy by Rank. The model performs consistently across all skill levels, with slight improvements in higher tiers.

4. Deep Learning Improvements

While traditional supervised learning achieved excellent results (88.6% accuracy), deep learning techniques could address specific limitations:

- **Neural Networks (MLP):** A Multi-Layer Perceptron could replace manual feature engineering by learning non-linear interactions from raw data, potentially capturing complex synergies between vision control and objective rates.
- **Sequence Modelling (RNN/LSTM):** The current dataset is static. An LSTM could process time-series data (e.g., gold at 10 mins, 20 mins) to predict outcomes in real-time, which would be valuable for live broadcast analytics.
- **Graph Neural Networks (GNN):** LoL is a team game. A GNN could model the 10 players as nodes and their interactions as edges, capturing team composition synergy rather than treating players as isolated rows in a spreadsheet. This would allow the model to pick up potential team character synergies (e.g., specific champion combinations that perform well together) that the current single-row analysis misses.

5. Discussion and Reflection

5.1 Interpretation of Findings

The results confirm that supervised learning effectively models esports outcomes, with Gradient Boosting proving that advanced ensemble methods yield tangible gains over traditional Logistic Regression. The performance gap (88.6% vs 84.7%) highlights that the path to victory is non-linear; a small lead in gold does not guarantee a win, but a lead in gold *combined* with high KDA creates a compound advantage that boosting algorithms map accurately.

The dominance of KDA in the feature importance analysis is a "reality check" for strategy (Silva et al. 2018). While coaches often preach "objectives over kills," the data suggests that at the individual level, your ability to kill enemies without dying is the strongest signal that you are winning. You cannot destroy a turret if you are dead.

5.2 Addressing Bias and Limitations

A critical reflection reveals inherent biases. First, the Rank Bias (dominance of Master tier) means the model learns high-level meta-strategies which may not fully apply to Iron players. Second, Outcome Bias is present in features like Kill_Differential; typically, you have more kills *because* you are winning, making it a lagging indicator.

These limitations are acknowledged. To address this Outcome Bias in future iterations, the model should be retrained using only data available at the 15-minute mark, removing end-game totals to create a true predictive tool rather than a descriptive one. Finally, the data is from a single patch (25.19). As the game developers change champion strength, this model would require retraining, necessitating a pipeline for continuous learning in a production environment.

5.3 Conclusion

This case study successfully demonstrated the application of supervised learning to esports analytics. By applying a "challenger" model in Gradient Boosting, we achieved a predictive accuracy of 88.6%, surpassing standard academic baselines. The analysis proved that while LoL is a strategy game, individual combat efficiency (KDA) remains the most reliable predictor of success. This report highlights the potential for data science to decode the complexities of competitive gaming, providing actionable intelligence for players and analysts alike.

References | AI was used to assist in Referencing

DataRobot 2023, *Introducing MLOps Champion/Challenger Models*, DataRobot Blog, viewed 20 November 2025, <https://www.datarobot.com/blog/introducing-mlops-champion-challenger-models/>.

Google 2025a, *Classification: ROC Curve and AUC*, Machine Learning Crash Course, viewed 27 November 2025, <https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc>.

Google 2025b, *Fairness: Types of Bias*, Machine Learning Crash Course, viewed 18 November 2025, <https://developers.google.com/machine-learning/crash-course/fairness/types-of-bias>.

IBM 2025, *What is Boosting?*, IBM Topics, viewed 20 November 2025, <https://www.ibm.com/think/topics/boosting>.

Nathan Small, 2023, *League of Legends Match History and Summoner Data (80k matches)*, Kaggle, viewed 9 November 2025, <https://www.kaggle.com/datasets/nathansmallcalder/lol-match-history-and-summoner-data-80k-matches>.

Scikit-learn Developers 2025, *Gradient Boosting classification*, Scikit-learn 1.6.1 documentation, viewed 25 November 2025, https://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_hist_grad_boosting_comparison.html.

Scikit-learn Developers 2025, *sklearn.linear_model.LogisticRegression*, Scikit-learn 1.6.1 documentation, viewed 10 November 2025, https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.

Appendix A: Code Annex

Source File: LoLML.py

Code Annex - LoLML.py

```
1  # League of Legends Match Outcome Prediction
2  # Machine Learning Classification Project | Note: AI was used to assist with
3  consolidating plots together for conciseness as well as general debugging
4
5  import pandas as pd
6  import numpy as np
7  import matplotlib
8  matplotlib.use('Agg') # Use non-interactive backend to avoid tkinter threading
9  issues
10 import matplotlib.pyplot as plt
11 import seaborn as sns
12 from sklearn.model_selection import train_test_split
13 from sklearn.preprocessing import StandardScaler, LabelEncoder
14 from sklearn.metrics import (accuracy_score, precision_score, recall_score,
15                               f1_score, roc_auc_score, confusion_matrix,
16                               roc_curve)
17 from sklearn.linear_model import LogisticRegression
18 from sklearn.tree import DecisionTreeClassifier
19 from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
20 import warnings
21 import os
22
23 warnings.filterwarnings('ignore')
24 np.random.seed(42)
25 os.makedirs('figures', exist_ok=True)
26
27 # LOAD DATA
28 # Import all CSV tables from the relational database
29 match_stats = pd.read_csv('MatchStatsTbl.csv')
30 champions = pd.read_csv('ChampionTbl.csv')
31 ranks = pd.read_csv('RankTbl.csv')
32 items = pd.read_csv('ItemTbl.csv')
33 summoner_match = pd.read_csv('SummonerMatchTbl.csv')
34 matches = pd.read_csv('MatchTbl.csv')
35 team_match = pd.read_csv('TeamMatchTbl.csv')
36
37 # MERGE TABLES
38 # Combine all relational tables using foreign keys to create single analysis
39 dataset
40 df = (match_stats
41       .merge(summoner_match, left_on='SummonerMatchFk',
42             right_on='SummonerMatchId', how='left')
43       .merge(matches, left_on='MatchFk', right_on='MatchId', how='left')
44       .merge(champions, left_on='ChampionFk', right_on='ChampionId',
45             how='left')
46       .merge(ranks, left_on='RankFk', right_on='RankId', how='left')
47       .merge(team_match, left_on='MatchFk', right_on='MatchFk', how='left'))
48
49
50
```

```

51 # Removing any rows that dont contain lane data as this represents players that
52 werent assigned a role.
53 # To me this means games that didn't start or corrupt data as players must be
54 assigned a role to be playing the game
55
56 # Remove records without valid lane assignment
57 df = df[df['Lane'].isin(['TOP', 'JUNGLE', 'MIDDLE', 'BOTTOM', 'SUPPORT'])]
58
59
60 # FEATURE ENGINEERING
61 # Create new features from raw data to improve model predictions
62
63 # Convert game duration from seconds to minutes for interpretability
64 df['GameDuration_mins'] = df['GameDuration'] / 60
65
66 # Per-minute metrics normalize performance by game length
67 df['CS_per_min'] = df['MinionsKilled'] / df['GameDuration_mins']
68 df['Gold_per_min'] = df['TotalGold'] / df['GameDuration_mins']
69 df['DmgDealt_per_min'] = df['DmgDealt'] / df['GameDuration_mins']
70 df['DmgTaken_per_min'] = df['DmgTaken'] / df['GameDuration_mins']
71 df['VisionScore_per_min'] = df['visionScore'] / df['GameDuration_mins']
72
73 # Combat metrics measure player effectiveness in fights
74 df['KDA'] = (df['kills'] + df['assists']) / df['deaths'].replace(0, 1) # Avoid
75 division by zero
76 df['Team_Kills'] = np.where(df['Win'] == df['BlueWin'], df['BlueKills'],
77 df['RedKills']) # Identify player's team
78 df['Kill_Participation'] = (df['kills'] + df['assists']) /
79 df['Team_Kills'].replace(0, 1)
80
81 # Efficiency metrics show how effectively resources are used
82 df['Dmg_per_Gold'] = df['DmgDealt'] / df['TotalGold'].replace(0, 1)
83
84 # Team differentials compare team performance directly
85 df['Kill_Differential'] = df['BlueKills'] - df['RedKills']
86 df['Baron_Differential'] = df['BlueBaronKills'] - df['RedBaronKills']
87 df['Dragon_Differential'] = df['BlueDragonKills'] - df['RedDragonKills']
88 df['Tower_Differential'] = df['BlueTowerKills'] - df['RedTowerKills']
89
90 # Objective metrics quantify strategic map control
91 df['Objective_Score'] = df['DragonKills'] + (df['BaronKills'] * 2) # Baron
92 weighted higher
93 item_cols = ['item1', 'item2', 'item3', 'item4', 'item5', 'item6']
94 df['Completed_Items'] = df[item_cols].notna().sum(axis=1) # Count non-empty
95 item slots
96 df['Mastery_log'] = np.log1p(df['CurrentMasteryPoints']) # Log transform for
97 normality
98
99 # Encode categorical variables into numeric format for ML models
100 le_lane = LabelEncoder()
101 df['Lane_Encoded'] = le_lane.fit_transform(df['Lane'].fillna('UNKNOWN'))
102 le_rank = LabelEncoder()
103 df['Rank_Encoded'] = le_rank.fit_transform(df['RankName'].fillna('UNRANKED'))
104
105 # Clean data by handling infinite values and missing data
106 df.replace([np.inf, -np.inf], np.nan, inplace=True) # Replace infinity with
107 NaN
108 numeric_cols = df.select_dtypes(include=[np.number]).columns
109 for col in numeric_cols:
110     if df[col].isnull().sum() > 0:
111

```

```

112         df[col].fillna(df[col].median(), inplace=True) # Fill missing with
113 median
114
115 # VISUALIZATIONS
116
117 # Plot 1: Win/Loss Distribution and Rank Distribution
118 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
119
120 # Show class balance in target variable
121 win_counts = df['Win'].value_counts()
122 axes[0].bar(['Loss', 'Win'], win_counts.values, color=['red', 'green'],
123 alpha=0.7)
124 axes[0].set_title('Match Outcomes', fontsize=14)
125 axes[0].set_ylabel('Number of Matches')
126 # Calculate percentages of win/loss in plot
127 total_matches = len(df)
128
129 for i, v in enumerate(win_counts.values):
130     percentage = (v / total_matches) * 100
131     # Add text with both Count and Percentage
132     axes[0].text(i, v, f'{v:,}\n({percentage:.1f}%)', ha='center', va='bottom',
133 fontweight='bold')
134
135 # Display most common ranks in dataset
136 rank_counts = df['RankName'].value_counts().head(10)
137 axes[1].barh(range(len(rank_counts)), rank_counts.values, color='steelblue')
138 axes[1].set_yticks(range(len(rank_counts)))
139 axes[1].set_yticklabels(rank_counts.index)
140 axes[1].set_title('Top 10 Ranks', fontsize=14)
141 axes[1].set_xlabel('Count')
142 axes[1].invert_yaxis()
143
144 plt.tight_layout()
145 plt.savefig('figures/01_outcomes_and_ranks.png', dpi=150)
146 plt.close()
147
148 # Plot 2: Feature Distributions by Win/Loss
149 # Compare how winning and losing players differ across key metrics
150 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
151 axes = axes.ravel()
152 features_to_plot = ['KDA', 'Gold_per_min', 'Kill_Participation',
153                     'CS_per_min', 'VisionScore_per_min', 'Dmg_per_Gold']
154
155 for idx, feature in enumerate(features_to_plot):
156     wins = df[df['Win'] == 1][feature].dropna()
157     losses = df[df['Win'] == 0][feature].dropna()
158
159     # Remove extreme outliers for cleaner visualization
160     wins = wins[wins < wins.quantile(0.99)]
161     losses = losses[losses < losses.quantile(0.99)]
162
163     axes[idx].hist(losses, bins=30, alpha=0.6, color='red', label='Loss')
164     axes[idx].hist(wins, bins=30, alpha=0.6, color='green', label='Win')
165     axes[idx].set_xlabel(feature)
166     axes[idx].set_ylabel('Frequency')
167     axes[idx].set_title(feature)
168     axes[idx].legend()
169     axes[idx].grid(alpha=0.3)
170
171 plt.tight_layout()
172 plt.savefig('figures/02_feature_distributions.png', dpi=150)

```

```

173 plt.close()
174
175 # Plot 3: Correlation Heatmap
176 # Show relationships between features and identify multicollinearity
177 features_for_corr = ['CS_per_min', 'Gold_per_min', 'DmgDealt_per_min',
178                     'KDA', 'Kill_Participation', 'VisionScore_per_min',
179                     'Kill_Differential', 'Baron_Differential',
180                     'Dragon_Differential', 'Tower_Differential',
181                     'Objective_Score', 'Completed_Items', 'Win']
182
183 corr_matrix = df[features_for_corr].corr()
184
185 plt.figure(figsize=(12, 10))
186 sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm',
187             center=0, square=True, linewidths=1, cbar_kws={"shrink": 0.8})
188 plt.title('Feature Correlation Matrix', fontsize=14)
189 plt.tight_layout()
190 plt.savefig('figures/03_correlation_heatmap.png', dpi=150)
191 plt.close()
192
193 # Plot 4: Win Rate by Lane
194 # Analyze if certain lanes have higher win rates
195 lane_stats = df.groupby('Lane').agg({'Win': ['mean', 'count']})
196 lane_stats.columns = ['WinRate', 'Count']
197 lane_stats = lane_stats.sort_values('WinRate', ascending=False)
198
199 plt.figure(figsize=(10, 6))
200 plt.bar(range(len(lane_stats)), lane_stats['WinRate'], color='teal', alpha=0.7)
201 plt.xticks(range(len(lane_stats)), lane_stats.index, rotation=45, ha='right')
202 plt.ylabel('Win Rate')
203 plt.xlabel('Lane')
204 plt.title('Win Rate by Lane')
205 plt.axhline(y=0.5, color='red', linestyle='--', linewidth=2, label='50%
206 Baseline')
207 plt.legend()
208 plt.grid(axis='y', alpha=0.3)
209
210 for i, (idx, row) in enumerate(lane_stats.iterrows()):
211     plt.text(i, row['WinRate'] + 0.01, f'n={row['Count']:.0f}',
212             ha='center', fontsize=9)
213
214 plt.tight_layout()
215 plt.savefig('figures/04_winrate_by_lane.png', dpi=150)
216 plt.close()
217
218 # Data Prep for Modelling
219
220 # Select features for model training
221 model_features = [
222     'CS_per_min', 'Gold_per_min', 'DmgDealt_per_min', 'DmgTaken_per_min',
223     'VisionScore_per_min', 'kills', 'deaths', 'assists', 'KDA',
224     'Kill_Participation', 'Dmg_per_Gold', 'Objective_Score',
225     'Kill_Differential', 'Baron_Differential', 'Dragon_Differential',
226     'Tower_Differential', 'Completed_Items', 'Mastery_log',
227     'Lane_Encoded', 'Rank_Encoded'
228 ]
229
230 # Separate features (X) from target variable (y)
231 x = df[model_features].copy()
232 y = df['Win'].copy()
233

```



```

234 # Splitting data into 80/20 test train data respectively
235 # Stratify ensures balanced class distribution in both sets
236 X_train, X_test, y_train, y_test = train_test_split(
237     X, y, test_size=0.2, random_state=42, stratify=y)
238
239 # Scale features to have mean=0 and std=1
240 # Important for distance-based algorithms and gradient descent
241 scaler = StandardScaler()
242 X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train),
243                               columns=model_features, index=X_train.index)
244 X_test_scaled = pd.DataFrame(scaler.transform(X_test),
245                              columns=model_features, index=X_test.index)
246
247 print(f"Training samples: {len(X_train):,}")
248 print(f"Test samples: {len(X_test):,}")
249 print(f"Features: {len(model_features)}")
250
251 # Training the Models
252 # Train multiple models to compare performance
253
254 models = {}
255 results = []
256
257 # Logistic Regression Model (lr)
258 # Linear model good for baseline and interpretability
259 lr_model = LogisticRegression(max_iter=1000, random_state=42, n_jobs=-1)
260 lr_model.fit(X_train_scaled, y_train)
261 lr_pred = lr_model.predict(X_test_scaled)
262 lr_proba = lr_model.predict_proba(X_test_scaled)[: , 1]
263
264 models['Logistic Regression'] = {'pred': lr_pred, 'proba': lr_proba}
265 results.append({
266     'Model': 'Logistic Regression',
267     'Accuracy': accuracy_score(y_test, lr_pred),
268     'Precision': precision_score(y_test, lr_pred),
269     'Recall': recall_score(y_test, lr_pred),
270     'F1-Score': f1_score(y_test, lr_pred),
271     'ROC-AUC': roc_auc_score(y_test, lr_proba)
272 })
273
274 # Decision Tree Model (dt)
275 # Non-linear model that creates interpretable rules
276 dt_model = DecisionTreeClassifier(max_depth=15, min_samples_split=100,
277                                   min_samples_leaf=50, random_state=42)
278 dt_model.fit(X_train, y_train)
279 dt_pred = dt_model.predict(X_test)
280 dt_proba = dt_model.predict_proba(X_test)[: , 1]
281
282 models['Decision Tree'] = {'pred': dt_pred, 'proba': dt_proba, 'model':
283 dt_model}
284 results.append({
285     'Model': 'Decision Tree',
286     'Accuracy': accuracy_score(y_test, dt_pred),
287     'Precision': precision_score(y_test, dt_pred),
288     'Recall': recall_score(y_test, dt_pred),
289     'F1-Score': f1_score(y_test, dt_pred),
290     'ROC-AUC': roc_auc_score(y_test, dt_proba)
291 })
292
293 # Random Forest Model (rf)
294 # Ensemble of decision trees reduces overfitting

```

```

295 rf_model = RandomForestClassifier(n_estimators=100, max_depth=15,
296                                 min_samples_split=100, min_samples_leaf=50,
297                                 random_state=42, n_jobs=-1, verbose=0)
298 rf_model.fit(X_train, y_train)
299 rf_pred = rf_model.predict(X_test)
300 rf_proba = rf_model.predict_proba(X_test)[:, 1]
301
302 models['Random Forest'] = {'pred': rf_pred, 'proba': rf_proba, 'model':
303 rf_model}
304 results.append({
305     'Model': 'Random Forest',
306     'Accuracy': accuracy_score(y_test, rf_pred),
307     'Precision': precision_score(y_test, rf_pred),
308     'Recall': recall_score(y_test, rf_pred),
309     'F1-Score': f1_score(y_test, rf_pred),
310     'ROC-AUC': roc_auc_score(y_test, rf_proba)
311 })
312
313 # Gradient Boosting Model (gb)
314 # Sequential ensemble that learns from previous tree errors
315 gb_model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1,
316                                     max_depth=5, min_samples_split=100,
317                                     min_samples_leaf=50, random_state=42,
318                                     verbose=0)
319 gb_model.fit(X_train, y_train)
320 gb_pred = gb_model.predict(X_test)
321 gb_proba = gb_model.predict_proba(X_test)[:, 1]
322
323 models['Gradient Boosting'] = {'pred': gb_pred, 'proba': gb_proba, 'model':
324 gb_model}
325 results.append({
326     'Model': 'Gradient Boosting',
327     'Accuracy': accuracy_score(y_test, gb_pred),
328     'Precision': precision_score(y_test, gb_pred),
329     'Recall': recall_score(y_test, gb_pred),
330     'F1-Score': f1_score(y_test, gb_pred),
331     'ROC-AUC': roc_auc_score(y_test, gb_proba)
332 })
333
334
335
336 # Model Comparison Plots
337
338 # Create results dataframe and save for reference
339 results_df = pd.DataFrame(results).round(4)
340 results_df = results_df.sort_values('Accuracy', ascending=False)
341 results_df.to_csv('model_results.csv', index=False)
342
343 # Plot 5: Model Performance Comparison
344 # Compare all models across multiple evaluation metrics
345 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
346 metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
347 colors = ['steelblue', 'coral', 'seagreen', 'orange']
348
349 for idx, (metric, color) in enumerate(zip(metrics, colors)):
350     ax = axes[idx // 2, idx % 2]
351     sorted_data = results_df.sort_values(metric, ascending=True)
352
353     ax.barh(range(len(sorted_data)), sorted_data[metric], color=color,
354             alpha=0.7)
355     ax.set_yticks(range(len(sorted_data)))

```

```

356     ax.set_yticklabels(sorted_data['Model'])
357     ax.set_xlabel(metric)
358     ax.set_title(f'{metric} Comparison')
359     ax.set_xlim(0, 1)
360     ax.grid(axis='x', alpha=0.3)
361
362     for i, val in enumerate(sorted_data[metric]):
363         ax.text(val + 0.01, i, f'{val:.3f}', va='center', fontsize=9)
364
365 plt.tight_layout()
366 plt.savefig('figures/05_model_metrics_comparison.png', dpi=150)
367 plt.close()
368
369 # Plot 6: ROC Curves
370 # Visualize true positive vs false positive rate tradeoff
371 plt.figure(figsize=(8, 6))
372
373 for model_name, model_data in models.items():
374     fpr, tpr, _ = roc_curve(y_test, model_data['proba'])
375     auc = roc_auc_score(y_test, model_data['proba'])
376     plt.plot(fpr, tpr, label=f'{model_name} (AUC={auc:.3f})', linewidth=2)
377
378 plt.plot([0, 1], [0, 1], 'k--', label='Random', linewidth=2)
379 plt.xlabel('False Positive Rate')
380 plt.ylabel('True Positive Rate')
381 plt.title('ROC Curves')
382 plt.legend(loc='lower right')
383 plt.grid(alpha=0.3)
384 plt.tight_layout()
385 plt.savefig('figures/06_roc_curves.png', dpi=150)
386 plt.close()
387
388 # Plot 7: Confusion Matrices
389 # Show prediction accuracy breakdown for all models
390 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
391 axes = axes.ravel()
392
393 for idx, (model_name, model_data) in enumerate(models.items()):
394     cm = confusion_matrix(y_test, model_data['pred'])
395
396     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[idx],
397                 cbar=False, square=True)
398 # Calculate accuracy specifically for this model
399     acc = accuracy_score(y_test, model_data['pred'])
400     axes[idx].set_title(f'{model_name}\nAccuracy: {acc:.3f}')
401     axes[idx].set_xlabel('Predicted')
402     axes[idx].set_ylabel('Actual')
403     axes[idx].set_xticklabels(['Loss', 'Win'])
404     axes[idx].set_yticklabels(['Loss', 'Win'])
405
406 plt.tight_layout()
407 plt.savefig('figures/07_confusion_matrices.png', dpi=150)
408 plt.close()
409
410 # Plot 8: Feature Importance
411 # Show which features matter most for tree-based models
412 fig, axes = plt.subplots(1, 3, figsize=(18, 6))
413
414 tree_models = ['Decision Tree', 'Random Forest', 'Gradient Boosting']
415 colors_fi = ['steelblue', 'seagreen', 'coral']
416

```

```

417 for idx, (model_name, color) in enumerate(zip(tree_models, colors_fi)):
418     importance_df = pd.DataFrame({
419         'Feature': model_features,
420         'Importance': models[model_name]['model'].feature_importances_
421     }).sort_values('Importance', ascending=False).head(10)
422
423     axes[idx].barh(range(len(importance_df)), importance_df['Importance'],
424                    color=color, alpha=0.7)
425     axes[idx].set_yticks(range(len(importance_df)))
426     axes[idx].set_yticklabels(importance_df['Feature'])
427     axes[idx].set_xlabel('Importance')
428     axes[idx].set_title(f'{model_name}\nTop 10 Features')
429     axes[idx].invert_yaxis()
430     axes[idx].grid(axis='x', alpha=0.3)
431
432 plt.tight_layout()
433 plt.savefig('figures/08_feature_importance.png', dpi=150)
434 plt.close()
435
436 # Plot 9: Rank Performance Analysis
437 # Evaluate model accuracy across different player ranks
438 df_with_pred = df.copy()
439
440 # 1. Generate Predictions
441 df_with_pred['RF_Prediction'] = rf_model.predict(X)
442
443 # 2. Determine if the prediction was correct (1 for yes, 0 for no)
444 df_with_pred['Correct'] = (df_with_pred['Win'] ==
445 df_with_pred['RF_Prediction']).astype(int)
446
447 # 3. Group by Rank and calculate Accuracy & Count
448 rank_performance = df_with_pred.groupby('RankName').agg({
449     'Correct': 'mean',          # Calculates Accuracy
450     'RF_Prediction': 'count'    # Calculates Sample Size
451 })
452
453 # 4. RENAME columns to match what the plotting loop expects ('Accuracy' and
454 'Count')
455 rank_performance = rank_performance.rename(columns={
456     'Correct': 'Accuracy',
457     'RF_Prediction': 'Count'    # <--- This fixes the KeyError
458 })
459
460 # 5. Sort by Accuracy
461 rank_performance = rank_performance.sort_values('Accuracy',
462 ascending=False).head(15)
463
464 # 6. Plotting
465 plt.figure(figsize=(12, 8))
466 bars = plt.barh(range(len(rank_performance)), rank_performance['Accuracy'],
467                 color='purple', alpha=0.7)
468 plt.yticks(range(len(rank_performance)), rank_performance.index)
469 plt.xlabel('Model Accuracy')
470 plt.title('Random Forest Accuracy by Rank (Top 15)')
471
472 # Add mean line
473 plt.axvline(x=rank_performance['Accuracy'].mean(), color='red',
474             linestyle='--', linewidth=2, label=f'Mean:
475 {rank_performance["Accuracy"].mean():.3f}')
476 plt.legend()
477 plt.grid(axis='x', alpha=0.3)

```

```

478 plt.gca().invert_yaxis()
479
    # Add text labels (This loop causes the error if 'Count' is missing)
    for i, (idx, row) in enumerate(rank_performance.iterrows()):
        plt.text(row['Accuracy'] + 0.005, i, f"n={row['Count']:, .0f}",
                va='center', fontsize=8)

plt.tight_layout()
plt.savefig('figures/09_rank_performance.png', dpi=150)
plt.close()

# Plot 10: Feature Correlation with Target
# Identify which features correlate most strongly with winning
win_correlations =
corr_matrix['Win'].drop('Win').abs().sort_values(ascending=False).head(10)

plt.figure(figsize=(10, 6))
colors_corr = ['green' if corr_matrix.loc[feat, 'Win'] > 0 else 'red'
               for feat in win_correlations.index]
plt.barh(range(len(win_correlations)), win_correlations.values,
        color=colors_corr, alpha=0.7)
plt.yticks(range(len(win_correlations)), win_correlations.index)
plt.xlabel('Absolute Correlation with Win')
plt.title('Top 10 Features Correlated with Match Outcome')
plt.gca().invert_yaxis()
plt.grid(axis='x', alpha=0.3)

for i, (feat, val) in enumerate(win_correlations.items()):
    actual_corr = corr_matrix.loc[feat, 'Win']
    plt.text(val + 0.01, i, f'{actual_corr:+.3f}', va='center', fontsize=9)

plt.tight_layout()
plt.savefig('figures/10_win_correlations.png', dpi=150)
plt.close()

```